

Loesungen — Blatt 11

Aufgaben: [\[\[numerik/exercises/11/num-exercise-11|Uebung 11\]\]](#) **PDF:** [\[\[numerik/exercises/11/num-solution-11.pdf|num-solution-11.pdf\]\]](#) **Quellcode:** [numerik/repos/numerik/blatt11/](#)

Inhaltsverzeichnis

- [\[\[#Aufgabe 1 — Trapez, Simpson und Gauss-Quadratur|Aufgabe 1 — Trapez, Simpson und Gauss-Quadratur\]\]](#)
 - [\[\[#Aufgabe 2 — Romberg- und Bulirsch-Extrapolation|Aufgabe 2 — Romberg- und Bulirsch-Extrapolation\]\]](#)
 - [\[\[#Aufgabe 3 — Adaptive Trapezregel|Aufgabe 3 — Adaptive Trapezregel\]\]](#)
-

Aufgabe 1 — Trapez, Simpson und Gauss-Quadratur

Zu berechnen ist $\int_0^1 \frac{1}{1+x^2} dx$ mit drei Quadraturregeln.

(a) **Exakter Wert**

$$\int_0^1 \frac{1}{1+x^2} dx = [\arctan x]_0^1 = \arctan 1 = \frac{\pi}{4} = 0.7853981634.$$

(b) **Summierte Trapez-Regel (8 Trapeze)**

Mit $n = 8$, $h = \frac{1}{8}$ und Stuetzstellen $x_i = ih$:

$$T_8 = h \left(\frac{1}{2}f(x_0) + \sum_{i=1}^7 f(x_i) + \frac{1}{2}f(x_8) \right).$$

```
def trapez_summiert(f, a, b, n):  
    x = np.linspace(a, b, n + 1)  
    h = (b - a) / n  
    y = f(x)  
    return h * (0.5 * y[0] + np.sum(y[1:-1]) + 0.5 * y[-1])
```

(c) **Summierte Simpson-Regel (4 Streifen)**

Mit $n = 4$ Streifen (also $n + 1 = 5$ Stuetzstellen), $h = \frac{1}{4}$:

$$S_4 = \frac{h}{3} \left(f(x_0) + 4(f(x_1) + f(x_3)) + 2f(x_2) + f(x_4) \right).$$

```
def simpson_summiert(f, a, b, n):
    x = np.linspace(a, b, n + 1)
    h = (b - a) / n
    y = f(x)
    return h / 3 * (y[0] + y[-1] + 4*np.sum(y[1:-1:2]) + 2*np.sum(y[2:-2:2]))
```

(d) Gauss-Verfahren $G_2(f)$

G_2 ist die 3-Punkt Gauss-Legendre-Regel auf $[-1, 1]$. Per Substitution $x = \frac{b-a}{2} \xi + \frac{a+b}{2}$, $dx = \frac{b-a}{2} d\xi$ auf $[0, 1]$:

$$G_2(f) = \frac{b-a}{2} \sum_{k=0}^2 \beta_k f\left(\frac{b-a}{2} \xi_k + \frac{a+b}{2}\right), \quad \xi_k \in \{-\sqrt{3/5}, 0, \sqrt{3/5}\}, \quad \beta = \left(\frac{5}{9}, \frac{8}{9}, \frac{5}{9}\right).$$

```
def gauss_g2(f, a, b):
    xi = np.array([-np.sqrt(3/5), 0.0, np.sqrt(3/5)])
    beta = np.array([5/9, 8/9, 5/9])
    x = 0.5 * (b - a) * xi + 0.5 * (a + b)
    return 0.5 * (b - a) * np.sum(beta * f(x))
```

Ergebnisse

Verfahren	Näherung	absoluter Fehler
exakt ($\pi/4$)	0.7853981634	—
Trapez (8 Trapeze)	0.7847471236	$6.510 \cdot 10^{-4}$
Simpson (4 Streifen)	0.7853921569	$6.007 \cdot 10^{-6}$
Gauss G_2	0.7852670350	$1.311 \cdot 10^{-4}$

[!tip] Merke Bei gleichem Aufwand liefert die **Simpson-Regel** (Genauigkeitsgrad 3) hier den kleinsten Fehler. G_2 nutzt nur **3 Funktionsauswertungen**, erreicht aber trotzdem eine bessere Genauigkeit als die Trapez-Regel mit 9 Auswertungen — Gauss-Quadratur mit $n + 1$ Knoten integriert Polynome bis Grad $2n + 1$ exakt.

Aufgabe 2 — Romberg- und Bulirsch-Extrapolation

Gesucht sind Näherungen von $\int_{-1}^1 \sin(\pi x^2) dx$ steigender Ordnung. Beide Verfahren extrapolieren die summierte **Trapez-Regel** $T(h)$ in h^2 mit dem Neville-Schema

$$T_{k,m} = T_{k,m-1} + \frac{T_{k,m-1} - T_{k-1,m-1}}{(n_k/n_{k-m})^2 - 1},$$

wobei n_k die Anzahl der Teilintervalle auf Stufe k ist. Der Unterschied liegt allein in der **Schrittweitenfolge**:

- **Romberg**: $n_k = 2^k = 1, 2, 4, 8, 16, 32, 64, 128$ (fortlaufende Halbierung; hier $(n_k/n_{k-m})^2 = 4^m$, das klassische $\frac{4^m T_{k,m-1} - T_{k-1,m-1}}{4^m - 1}$).
- **Bulirsch**: $n_k = 1, 2, 3, 4, 6, 8, 12, 16$ (Bulirsch-Folge, langsamer wachsend $\rightarrow$ weniger Auswertungen auf hohen Stufen).

Die Diagonalelemente $T_{k,k}$ liefern die Naehierungen der Ordnung $2(k+1)$.

```
def extrapolation(f, a, b, seq):
    K = len(seq)
    T = np.zeros((K, K))
    for k in range(K):
        T[k, 0] = trapez(f, a, b, seq[k])
        for m in range(1, k + 1):
            r = (seq[k] / seq[k - m])**2
            T[k, m] = T[k, m-1] + (T[k, m-1] - T[k-1, m-1]) / (r - 1)
    return T
```

Ergebnisse (Referenz `scipy.integrate.quad` = 1.009709188227)

Ordnung	Romberg $T_{k,k}$	Fehler	Bulirsch $T_{k,k}$	Fehler
2	0.000000000000	$1.01 \cdot 10^0$	0.000000000000	$1.01 \cdot 10^0$
4	0.000000000000	$1.01 \cdot 10^0$	0.000000000000	$1.01 \cdot 10^0$
6	1.005662977688	$4.05 \cdot 10^{-3}$	0.923454386979	$8.63 \cdot 10^{-2}$
8	1.025042824589	$1.53 \cdot 10^{-2}$	1.111359737170	$1.02 \cdot 10^{-1}$
10	1.009493395968	$2.16 \cdot 10^{-4}$	1.013366654725	$3.66 \cdot 10^{-3}$
12	1.009707587534	$1.60 \cdot 10^{-6}$	1.007770275683	$1.94 \cdot 10^{-3}$
14	1.009709194163	$5.94 \cdot 10^{-9}$	1.009716145508	$6.96 \cdot 10^{-6}$
16	1.009709188227	$3.07 \cdot 10^{-13}$	1.009713408770	$4.22 \cdot 10^{-6}$

[!warning] Achtung Die ersten Stufen sind exakt 0: Der Integrand $\sin(\pi x^2)$ verschwindet an $x = -1, 0, 1$ (denn $\sin(\pi) = \sin(0) = 0$), sodass die groben Trapezsummen bei zu wenigen Stuetzstellen genau die Nullstellen abtasten. Erst ab genuegend feinem Gitter wird die Oszillation der Funktion aufgeloeset.

[!tip] Merke Beide Verfahren konvergieren; die **Romberg**-Folge erreicht hier bei Ordnung 16 nahezu Maschinengenauigkeit ($3 \cdot 10^{-13}$), waehrend **Bulirsch** mit der langsamer wachsenden Knotenfolge weniger Funktionsauswertungen benoetigt, da fuer bei gleicher Stufenzahl etwas groebere Naehierungen ($4 \cdot 10^{-6}$) liefert.

Aufgabe 3 — Adaptive Trapezregel

Zu berechnen ist $\int_0^4 \frac{3x}{x+1} dx$.

(a) Exakter Wert

Mit $\frac{3x}{x+1} = 3 - \frac{3}{x+1}$:

$$\int_0^4 \frac{3x}{x+1} dx = \left[3x - 3 \ln(x+1) \right]_0^4 = 12 - 3 \ln 5 = 7.1716862627.$$

(b) Adaptive Trapezregel

Auf jedem Teilintervall $[a_i, b_i]$ werden das einfache Trapez T und das halbierte Trapez $\tilde{T}$ (mit Mittelpunkt) berechnet. Der Fehlerschaetzer $\tilde{F} = \frac{|\tilde{T}-T|}{3}$ entscheidet: ist $\tilde{F} < 0.05$, wird $\tilde{T}$ akzeptiert, sonst wird das Intervall halbiert und rekursiv weiterbearbeitet. Start: **ein** Trapez auf $[0, 4]$.

```
def adaptiv(f, a, b, tol, intervale):
    T = (b - a) / 2 * (f(a) + f(b))           # ein Trapez
    m = 0.5 * (a + b)
    Tt = (b - a) / 4 * (f(a) + 2*f(m) + f(b)) # zwei Trapeze
    F = abs(Tt - T) / 3
    if F < tol:
        intervale.append((a, b, Tt, F))
        return Tt
    return adaptiv(f, a, m, tol, intervale) + adaptiv(f, m, b, tol, intervale)
```

Ergebnisse

Das Startintervall $[0, 4]$ wird zunaechst halbiert; das linke Stueck $[0, 2]$ nochmals, das rechte $[2, 4]$ wird sofort akzeptiert. Es verbleiben **3** akzeptierte Teilintervalle:

Teilintervall $[a_i, b_i]$	$\tilde{T}$	$\tilde{F}$
$[0, 1]$	0.875000	0.04167
$[1, 2]$	1.775000	0.00833
$[2, 4]$	4.450000	0.01667

Naeherung = $0.875+1.775+4.450 = 7.1000000000$, $|\text{Fehler}| = |7.1-7.1716862627| = 7.169 \cdot 10^{-2}$.

[!success] Best Practice Die adaptive Verfeinerung konzentriert die Stuetzstellen dort, wo die Kruemmung gross ist: nahe $x = 0$ ist $\frac{3x}{x+1}$ am staerksten gekruemmt, weshalb $[0, 2]$ weiter unterteilt wird, waehrend das flachere $[2, 4]$ mit einem einzigen Trapezpaar auskommt. So wird der Aufwand dem lokalen Fehler angepasst, statt gleichmaessig zu verfeinern.

[!warning] Achtung Das Abbruchkriterium $\tilde{F} < 0.05$ ist eine **lokale** Fehlerschranke pro Teilintervall. Der tatsaechliche Gesamtfehler ($7.2 \cdot 10^{-2}$) kann daher etwas groesser ausfallen als die einzelnen Schranken — die $\tilde{F}$ -Werte schaetzen den Fehler der jeweiligen Trapezregel, nicht den akkumulierten Gesamtfehler.